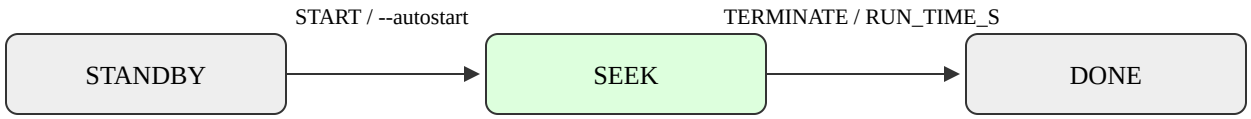
propPitch

Collective prop pitch held where the plane accelerates fastest. `./run.sh --pilot prop`

What it flies

Only the propellers. The pilot (or a harness) flies the plane. `propPitch/autopilot.py` is the one kRPC-aware module; `seeker.py` holds the law and no kRPC.



STANDBY → SEEK: the takeover

- Finds every robotic rotor (`parts.robotic_rotors`). Below each, every part with a control-surface module carrying a `Deploy Angle` field is a blade.
- Disables every KAL robotic controller (`controllerEnabled`), which would otherwise rewrite the blades each frame.
- Each rotor: motor engaged, unlocked, torque limit `ROTOR_TORQUE_PCT`, RPM limit `ROTOR_RPM`. Their earlier values are kept.
- The seeker's centre starts at the blades' mean Deploy Angle (or `INITIAL_PITCH_DEG` ≥ 0).

SEEK: every tick (`LOOP_SLEEP_S`, game time)

1. One batched read: aero force in the vessel frame, mass, speed, height, vertical speed, acceleration, nose direction, position, EC, and each rotor's RPM.
2. **Objective** J = forward (vessel y) aero force ÷ mass. That is propeller thrust minus airframe drag, as an acceleration. The log also gives `acc=`, the measured acceleration along the nose less gravity, as a check.
3. Every `REASSERT_S`: the KALs' Enabled flag and the rotors' limits are read back, re-set if anything changed them, and logged.
4. The seeker takes (t, J) and returns the pitch. Every blade's Deploy Angle is written when it moves ≥ 0.01°.

The seeker

Command: $p(t) = c + A \sin \omega t$, with A = `DITHER_AMP_DEG` and period `DITHER_PERIOD_S`, clamped to [`PITCH_MIN_DEG`, `PITCH_MAX_DEG`].

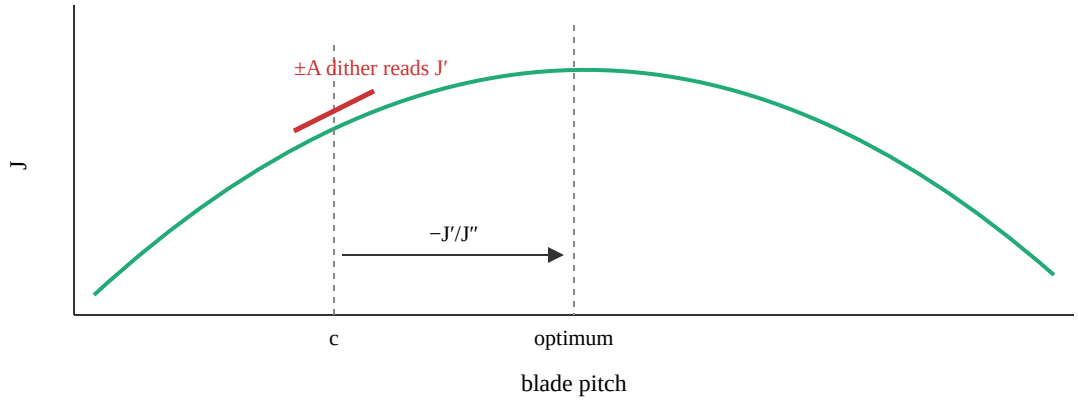
Over the last `WINDOW_PERIODS` periods, least squares fits

$$J = a + b\tau + e\tau^2 + s \sin \varphi + q \cos \varphi + s_2 \sin 2\varphi + c_2 \cos 2\varphi, \quad \varphi = \omega t - \text{lag}$$

τ is time in periods. The trend terms take up the airspeed changing under the window. The rest follows from expanding $J(c + A \sin)$:

quantity	from

slope J'	s / A
slope's t-statistic	s / se(s)
curvature J''	$-4\,c_2$ / A^2
response lag	atan2($-q$, s) (logged; `PHASE_LAG_DEG` is the assumed one)



The dither reads the local slope; the second harmonic reads the bend; their ratio is the distance to the top.

Moving the centre, each tick of length dt:

- A fit whose curvature is negative by more than `CURVATURE_MIN_T` standard errors updates a smoothed curvature (time constant `CURVATURE_TAU_S`).
- **With** a smoothed curvature: Newton. The rate is ($-J'/J''$) / `NEWTON_TAU_S`, capped at ± `RATE_MAX_DEG_S`.
- **Without** one: the rate is `RATE_MAX_DEG_S` · tanh(t-stat / `SLOPE_T_SCALE`). It is fast when the slope's sign is certain, however small the slope.
- No fit until the window spans a full period.

SEEK → DONE: the hand-back

TERMINATE, `RUN_TIME_S` or an error. The rotors' RPM, torque limit and motor state go back to what they were, and the KALs are re-enabled (they take the blades back from the throttle). The log ends with the loop-rate and RPC-cost lines.

Log line

`SEEK spd alt vs pitch ctr J acc rpm aoa ec slope t curv/t lag trend rate [newton]`, every `LOG_INTERVAL_UT` game seconds.